

GANDHINAGAR UNIVERSITY

Gandhinagar Institute of Technology

Diploma CE / CSE / IT | Semester 4 | Even Term 2025-26

Subject Code: 101240401

Advanced Object Oriented Programming (Java)

COMPLETE QUESTION BANK ANSWERS

All 64 Questions | Question + Concept + Structured Answer

Units 1, 2 & 3

HOW TO READ THIS DOCUMENT

Question	Concept	Structured Answer	Java Code
----------	---------	-------------------	-----------

Total: 64 Questions across 3 Units | Marks indicated in each question header

UNIT 1 — Introduction to Java & OOP Basics (Q1 – Q30)

Q1 What is Java?

[1 Mark]

■ CONCEPT

Java is a high-level, class-based, object-oriented programming language developed by Sun Microsystems in 1995.

Core idea: Write Once, Run Anywhere (WORA) — code compiles to bytecode that runs on any machine with a JVM.

- Platform Independent (WORA principle)
- Object-Oriented: everything is in classes/objects
- Secure, Robust, Multithreaded
- Used for: Web apps, Android apps, enterprise software

■ STRUCTURED ANSWER

■ Definition:

Java is a high-level, object-oriented, platform-independent programming language developed by Sun Microsystems (now Oracle) in 1995.

■ Key Features:

- Platform Independent — bytecode runs on any JVM
- Object Oriented — based on classes and objects
- Secure — no pointers, strong type checking
- Multithreaded — built-in thread support
- Robust — exception handling, garbage collection

Q2 What is JVM?

[1 Mark]

■ CONCEPT

JVM (Java Virtual Machine) is the engine that runs Java programs.

Java source (.java) → compiler (javac) → bytecode (.class) → JVM executes it.

- JVM is platform-specific but bytecode is platform-independent
- This is why Java achieves WORA

■ STRUCTURED ANSWER

■ Definition:

JVM stands for Java Virtual Machine. It is an abstract machine that provides a runtime environment to execute Java bytecode.

■ Working Process:

- Step 1: javac compiles .java source to .class (bytecode)
- Step 2: JVM reads and interprets bytecode
- Step 3: JVM translates bytecode to native machine code
- Different OS has different JVM, but same bytecode runs on all

Q3

What is a class in Java?

[1 Mark]

■ CONCEPT

A class is a blueprint or template for creating objects.

Think of it as a house plan — the plan defines structure, objects are actual houses built from it.

- A class defines: variables (state) + methods (behaviour)

■ STRUCTURED ANSWER

■ Definition:

A class is a user-defined blueprint from which objects are created. It encapsulates data (fields) and behaviour (methods).

■ Syntax:

```
class ClassName {  
    // instance variables (state)  
    String name;  
    int age;  
    // methods (behaviour)  
    void display() { System.out.println(name); }  
}
```

Q4

What is an object?

[1 Mark]

■ CONCEPT

An object is a real-world entity — an instance of a class with actual values.

If Car is the class, then myCar (colour=red, speed=100) is an object.

- Every object has: state (data) + behaviour (methods) + identity (unique reference)

■ STRUCTURED ANSWER

■ Definition:

An object is an instance of a class. It occupies memory and has state (values in fields) and behaviour (methods it can call).

■ Creating an Object:

```
ClassName obj = new ClassName(); // 'new' allocates memory  
obj.fieldName = value; // set state  
obj.methodName(); // invoke behaviour  
  
// Example:  
Car myCar = new Car();  
myCar.colour = "Red";  
myCar.drive();
```

Q5

What is type casting?

[1 Mark]

■ **CONCEPT**

Type casting = converting a variable from one data type to another.

- Widening (Implicit): smaller → larger type, automatic, no data loss
- Narrowing (Explicit): larger → smaller type, manual using (type), may lose data

■ **STRUCTURED ANSWER**

■ **Two Types of Type Casting:**

■ **1. Widening (Implicit/Automatic):**

```
int a = 10;
double b = a; // int automatically becomes 10.0
// Order: byte -> short -> int -> long -> float -> double
```

■ **2. Narrowing (Explicit/Manual):**

```
double x = 9.99;
int y = (int) x; // y = 9 (decimal part lost!)
// Must write (type) before the value
```

Q6

What is the use of the 'this' keyword?

[1 Mark]

■ **CONCEPT**

'this' refers to the current object inside a class.

- Used to distinguish instance variables from method parameters with same name
- Used to call another constructor: this() or this(args)
- Used to pass current object as argument to another method

■ STRUCTURED ANSWER

■ Three Uses of 'this':

```
class Student {  
    String name; int age;  
  
    // Use 1: Resolve naming conflict  
    Student(String name, int age) {  
        this.name = name; // this.name = field, name = param  
        this.age = age;  
    }  
    // Use 2: Call another constructor  
    Student() {  
        this("Unknown", 0); // calls above constructor  
    }  
    // Use 3: Pass current object  
    void show(Student s) { System.out.println(s.name); }  
    void call() { show(this); }  
}
```

Q7

What is method overloading?

[1 Mark]

■ CONCEPT

Method overloading = same method name, different parameter lists in the same class.

The compiler decides which version to call based on arguments — compile-time polymorphism.

- Parameters can differ by: number, type, or order

■ STRUCTURED ANSWER

■ Definition:

Method overloading allows multiple methods with the same name but different parameters in the same class.

■ Example:

```
class Demo {  
    void show(int a) { System.out.println("int: " + a); }  
    void show(double a) { System.out.println("double: " + a); }  
    void show(int a, int b) { System.out.println("sum: " + (a+b)); }  
    void show(String a) { System.out.println("String: " + a); }  
}  
// Java picks correct version based on argument types
```

Q8**What is a constructor?****[1 Mark]****■ CONCEPT**

A constructor is a special method called automatically when an object is created.

- Same name as class, no return type
- Used to initialise object's fields
- If not defined, Java provides a default (no-arg) constructor

■ STRUCTURED ANSWER**■ Types of Constructors:**

```
class Car {  
    String brand; int year;  
  
    // Default constructor (no parameters)  
    Car() {  
        brand = "Unknown"; year = 2000;  
    }  
  
    // Parameterised constructor  
    Car(String brand, int year) {  
        this.brand = brand; this.year = year;  
    }  
}  
  
Car c1 = new Car(); // calls default  
Car c2 = new Car("Toyota", 2023); // calls parameterised
```

Q9**Name any two primitive data types in Java with examples.****[2 Marks]****■ CONCEPT**

Java has 8 primitive types: byte, short, int, long, float, double, char, boolean.

- int: whole numbers, 4 bytes, range -2 billion to +2 billion
- double: decimal numbers, 8 bytes, 15-16 significant digits

■ STRUCTURED ANSWER

■ Two Primitive Data Types:

```
// 1. int – stores whole numbers (4 bytes)
int age = 21;
int marks = 95;
int year = 2025;

// 2. double – stores decimal numbers (8 bytes)
double price = 1299.99;
double pi = 3.14159;
double cgpa = 8.75;

// Other primitives for reference:
// byte(1), short(2), long(8), float(4), char(2), boolean(1)
```

Q1 What is the difference between == operator and .equals()
0. method?

[2 Marks]

■ CONCEPT

== compares memory addresses (references) — are they the same object?

.equals() compares content/value — do they have the same data?

- For primitives, use == | For Strings/objects, use .equals()

■ STRUCTURED ANSWER

■ == vs .equals():

```
String a = new String("hello");
String b = new String("hello");

System.out.println(a == b); // false (different objects in memory)
System.out.println(a.equals(b)); // true (same content)

// String literals share pool – == may return true:
String x = "hello";
String y = "hello";
System.out.println(x == y); // true (same pool reference)
System.out.println(x.equals(y)); // true (same content)
```

Q1
1. What is the use of the 'static' keyword in Java?

[2 Marks]

■ CONCEPT

'static' means the member belongs to the CLASS, not to any specific object.

- static variable: shared among ALL objects (one copy only)
- static method: callable without creating an object
- static block: runs once when class is loaded

■ STRUCTURED ANSWER

■ Applications of 'static':

```
class Counter {
    static int count = 0; // shared – ONE copy for all objects
    String name;

    Counter(String name) {
        this.name = name;
        count++; // increments for every object created
    }

    // static method – called on class, not object
    static void showCount() {
        System.out.println("Total objects: " + count);
    }
}

Counter c1 = new Counter("A");
Counter c2 = new Counter("B");
Counter.showCount(); // Output: Total objects: 2
```

Q1 What is the difference between Procedure-Oriented and
2. Object-Oriented Programming?

[3 Marks]

■ CONCEPT

POP focuses on functions; OOP focuses on objects combining data + behaviour.

- POP: top-down approach, data is global, less secure (C, Pascal)
- OOP: bottom-up approach, data is encapsulated, more secure (Java, C++)

■ STRUCTURED ANSWER

■ Comparison Table:

Feature | POP (C, Pascal) | OOP (Java, C++)
Approach | Top-down | Bottom-up
Basic Unit | Functions | Objects + Classes
Data Access | Global, less secure | Encapsulated (private)
Inheritance | Not supported | Supported
Overloading | Not supported | Supported
Code Reuse | Limited | High (via inheritance)

■ OOP Advantages over POP:

- Encapsulation hides data — better security
- Inheritance reduces code duplication
- Polymorphism makes code flexible and extendable
- Better suited for large, complex applications

Q1
3.

What is JDK? How is it different from JRE?

[2 Marks]

■ CONCEPT

JDK = Java Development Kit — complete package for DEVELOPERS (write + compile + run)

JRE = Java Runtime Environment — only for RUNNING Java programs

- JDK includes JRE; JRE does NOT include compiler

■ STRUCTURED ANSWER

■ JDK vs JRE:

JDK contains:

javac (Java Compiler — .java -> .class)

javadoc (Documentation generator)

jdb (Java Debugger)

jar (Archive tool)

+ JRE (everything below)

JRE contains:

JVM (Java Virtual Machine — runs .class)

Core libraries (java.lang, java.util, java.io ...)

// Who needs what?

// Developers -> need JDK

// End-users -> need only JRE

Q1
4.

What is the difference between int and Integer in Java?

[2 Marks]

■ CONCEPT

int is a primitive type — stores value directly, very fast, cannot be null.

Integer is a Wrapper class (object) — wraps int in an object.

- Integer needed for Collections (ArrayList, HashMap) which require objects
- Java auto-boxes/unboxes between int and Integer automatically

■ STRUCTURED ANSWER

■ int vs Integer:

```
// int: primitive, stored on stack
int x = 10;

// Integer: object, stored on heap
Integer y = new Integer(10); // explicit boxing
Integer z = 10; // auto-boxing (Java 5+)
int p = z; // auto-unboxing

// Integer utility methods:
int max = Integer.MAX_VALUE; // 2147483647
int min = Integer.MIN_VALUE; // -2147483648
int num = Integer.parseInt("42"); // String -> int
String s = Integer.toString(5); // "5"

// ArrayList NEEDS Integer, not int:
ArrayList list = new ArrayList<>(); // OK
// ArrayList list = ...; // COMPILER ERROR
```

Q1
5.

What is a wrapper class? Give two examples.

[2 Marks]

■ CONCEPT

A wrapper class wraps a primitive data type inside an object.

Purpose: Allow primitives to be used in Collections (which need objects).

- Boxing: primitive → wrapper | Unboxing: wrapper → primitive
- Java 5+ supports auto-boxing/unboxing

■ STRUCTURED ANSWER

■ Definition + Examples:

```
// Every primitive has a wrapper:
// byte->Byte, short->Short, int->Integer, long->Long
// float->Float, double->Double, char->Character, boolean->Boolean

// Example 1: Integer wraps int
Integer a = 42; // auto-boxing
int b = a; // auto-unboxing
System.out.println(Integer.toBinaryString(42)); // 101010
System.out.println(Integer.MAX_VALUE); // 2147483647

// Example 2: Double wraps double
Double d = 3.14;
System.out.println(d.compareTo(3.0)); // 1 (3.14 > 3.0)
System.out.println(Double.isNaN(0.0/0.0)); // true
```

Q1
6.

What is the use of 'final' keyword in Java? Give one example.

[2 Marks]

■ CONCEPT

'final' restricts modification — cannot be changed after first assignment.

- final variable: constant (value cannot change)
- final method: cannot be overridden in subclass
- final class: cannot be extended/subclassed

■ STRUCTURED ANSWER

■ Three Uses of 'final':

```
// 1. final variable – constant
final double PI = 3.14159;
// PI = 3.0; // COMPILE ERROR: cannot reassign

// 2. final method – cannot override
class Parent {
    final void display() { System.out.println("Parent"); }
}
class Child extends Parent {
    // void display() {} // ERROR: cannot override final
}

// 3. final class – cannot subclass
final class MyClass { }
// class Sub extends MyClass {} // ERROR

// Note: String is a final class in Java!
```

Q1
7.

What is scope of a variable in Java?

[2 Marks]

■ **CONCEPT**

Scope = the region of code where a variable can be accessed.

- Local: inside a method/block only
- Instance: inside the class, accessible through object
- Static/Class: accessible via class name, no object needed

■ **STRUCTURED ANSWER**

■ **Types of Variable Scope:**

```
class ScopeDemo {
    static int classVar = 100; // class scope
    int instanceVar = 50; // instance scope

    void method() {
        int localVar = 10; // local scope (method only)
        {
            int blockVar = 5; // block scope
            System.out.println(blockVar); // OK inside block
        }
        // System.out.println(blockVar); // ERROR: out of scope
        System.out.println(localVar); // OK
        System.out.println(instanceVar); // OK via 'this'
        System.out.println(classVar); // OK
    }
}
```

Q1
8.

What is type conversion? Give an example of implicit and explicit conversion.

[2 Marks]

■ **CONCEPT**

Type conversion = changing a value from one data type to another.

- Implicit (Widening): automatic, smaller→larger, no data loss
- Explicit (Narrowing): manual with (type) cast, larger→smaller, may lose data

■ STRUCTURED ANSWER

■ Type Conversion Examples:

```
// IMPLICIT (Widening) – Java handles automatically
int num = 100;
long l = num; // int -> long (automatic)
double d = num; // int -> double (automatic)
System.out.println(d); // 100.0

// EXPLICIT (Narrowing) – programmer must cast
double price = 99.99;
int p = (int) price; // double -> int (manual)
System.out.println(p); // 99 (0.99 is LOST!)

// Widening order (all automatic):
// byte -> short -> int -> long -> float -> double
```

Q1 What is an array in Java? How is a 1D array declared and
9. initialized?

[2 Marks]

■ CONCEPT

An array stores multiple values of the SAME type in a fixed-size contiguous block.

- Index starts at 0, ends at length-1
- Size is fixed once created

■ STRUCTURED ANSWER

■ Array Declaration and Initialization:

```
// Method 1: Declare, then initialize
int[] marks; // declaration
marks = new int[5]; // initialization (default 0)

// Method 2: Declare + Initialize together
int[] scores = new int[5]; // all zeros initially
scores[0] = 90;
scores[1] = 85;

// Method 3: Declare + Initialize with values
int[] result = {90, 85, 78, 92, 88};

// Access & Length:
System.out.println(result[0]); // 90 (first)
System.out.println(result[4]); // 88 (last)
System.out.println(result.length); // 5
```

Q2
0.

What is the difference between break and continue statements?

[2 Marks]

■ **CONCEPT**

break: exits the loop/switch entirely — no more iterations.

continue: skips current iteration, jumps to next — loop continues.

- break ends the loop; continue skips one round

■ **STRUCTURED ANSWER**

■ **break vs continue with Output:**

```
// break – stops the loop at i=5
for (int i = 1; i <= 10; i++) {
    if (i == 5) break;
    System.out.print(i + " ");
}
// Output: 1 2 3 4

// continue – skips i=5, loop goes on
for (int i = 1; i <= 10; i++) {
    if (i == 5) continue;
    System.out.print(i + " ");
}
// Output: 1 2 3 4 6 7 8 9 10
```

Q2
1.

Explain the four basic principles of OOP: Encapsulation, Inheritance, Polymorphism, and Abstraction with examples.

[5 Marks]

■ **CONCEPT**

OOP is built on 4 pillars:

1. Encapsulation — hide data, expose only what is needed
2. Inheritance — child class reuses parent class code
3. Polymorphism — same name, different behaviour
4. Abstraction — hide implementation, show only interface

■ STRUCTURED ANSWER

■ 1. Encapsulation (Data Hiding):

```
class BankAccount {  
    private double balance; // hidden  
    public void deposit(double a) { balance += a; }  
    public double getBalance() { return balance; }  
}
```

■ 2. Inheritance (Code Reuse):

```
class Animal { void eat() { System.out.println("Eating"); } }  
class Dog extends Animal {  
    void bark() { System.out.println("Woof!"); }  
}  
Dog d = new Dog(); d.eat(); d.bark(); // eat() inherited
```

■ 3. Polymorphism (Many Forms):

```
class Shape { void area() { } }  
class Circle extends Shape { void area() { System.out.println("PI*r*r"); } }  
class Rectangle extends Shape { void area() { System.out.println("L*B"); } }  
Shape s = new Circle(); s.area(); // Circle's area at runtime
```

■ 4. Abstraction (Hide Details):

```
abstract class Vehicle {  
    abstract void start(); // must implement  
    void fuel() { System.out.println("Refuelling..."); }  
}  
class Car extends Vehicle {  
    void start() { System.out.println("Car started"); }  
}
```

Q2 Write a Java program to demonstrate method overloading with a
2. Calculator class.

[6 Marks]

■ CONCEPT

Method Overloading: same method name, different parameters, same class.

Compiler selects the right method at compile time = static polymorphism.

- Parameters differ by: number, type, or order of arguments

■ STRUCTURED ANSWER

■ Calculator class with overloaded methods:

```
class Calculator {  
    // add: two integers  
    int add(int a, int b) { return a + b; }  
  
    // add: two doubles  
    double add(double a, double b) { return a + b; }  
  
    // add: three integers  
    int add(int a, int b, int c) { return a + b + c; }  
  
    // multiply: two integers  
    int multiply(int a, int b) { return a * b; }  
  
    // multiply: two doubles  
    double multiply(double a, double b) { return a * b; }  
  
    public static void main(String[] args) {  
        Calculator calc = new Calculator();  
        System.out.println(calc.add(5, 10)); // 15  
        System.out.println(calc.add(3.5, 2.5)); // 6.0  
        System.out.println(calc.add(1, 2, 3)); // 6  
        System.out.println(calc.multiply(4, 5)); // 20  
        System.out.println(calc.multiply(2.5, 4.0)); // 10.0  
    }  
}
```

Q2 Write a Java program to demonstrate constructor overloading
3. with a Student class.

[6 Marks]

■ CONCEPT

Constructor Overloading: multiple constructors with different parameters in same class.

Allows creating objects with varying amounts of data.

- this() inside constructor calls another constructor of same class

■ STRUCTURED ANSWER

■ Student class with overloaded constructors:

```
class Student {
    String name; int age; double marks; String course;

    Student() { // Default
        name="Unknown"; age=0; marks=0; course="N/A";
    }
    Student(String name) { // Name only
        this(); this.name = name;
    }
    Student(String name, int age) { // Name + Age
        this.name=name; this.age=age;
        marks=0; course="N/A";
    }
    Student(String name, int age, double marks, String course) {
        this.name=name; this.age=age;
        this.marks=marks; this.course=course;
    }
    void display() {
        System.out.println(name+", "+age+", "+marks+", "+course);
    }
    public static void main(String[] args) {
        new Student().display();
        new Student("Alice").display();
        new Student("Bob", 20).display();
        new Student("Carol", 21, 88.5, "CE").display();
    }
}
```

Q2 Explain the String class in Java. Write a program demonstrating
4. any five String methods.

[7 Marks]

■ CONCEPT

String is a class (not primitive) in java.lang. Strings are IMMUTABLE — once created, value cannot change.

Every String operation creates a new String object.

- Key methods: length(), charAt(), substring(), toUpperCase(), toLowerCase(), equals(), contains(), replace(), split(), indexOf()

■ STRUCTURED ANSWER

■ String class with 8 important methods:

```
public class StringDemo {
    public static void main(String[] args) {
        String s = "Hello Java World";

        // 1. length()
        System.out.println(s.length()); // 16

        // 2. toUpperCase() / toLowerCase()
        System.out.println(s.toUpperCase()); // HELLO JAVA WORLD
        System.out.println(s.toLowerCase()); // hello java world

        // 3. charAt(index)
        System.out.println(s.charAt(0)); // H
        System.out.println(s.charAt(6)); // J

        // 4. substring(start, end)
        System.out.println(s.substring(6, 10)); // Java

        // 5. contains()
        System.out.println(s.contains("Java")); // true

        // 6. replace()
        System.out.println(s.replace('l', 'L')); // HeLLo Java WorLd

        // 7. equals() and equalsIgnoreCase()
        System.out.println(s.equals("hello java world")); // false
        System.out.println(s.equalsIgnoreCase("HELLO JAVA WORLD")); // true

        // 8. split()
        String[] words = s.split(" ");
        for(String w : words) System.out.print(w+" | ");
        // Hello | Java | World |
    }
}
```

Q2
5.

Write a Java program to add two 3x3 matrices.

[5 Marks]

■ CONCEPT

Matrix addition: add corresponding elements — $\text{result}[i][j] = A[i][j] + B[i][j]$.

- Both matrices must be same size
- Use nested loops: outer for rows, inner for columns

■ STRUCTURED ANSWER

■ 3x3 Matrix Addition Program:

```
public class MatrixAddition {  
    public static void main(String[] args) {  
        int[][] A = { {1,2,3}, {4,5,6}, {7,8,9} };  
        int[][] B = { {9,8,7}, {6,5,4}, {3,2,1} };  
        int[][] C = new int[3][3];  
  
        // Add: C[i][j] = A[i][j] + B[i][j]  
        for (int i = 0; i < 3; i++)  
            for (int j = 0; j < 3; j++)  
                C[i][j] = A[i][j] + B[i][j];  
  
        // Print result  
        System.out.println("Result Matrix:");  
        for (int i = 0; i < 3; i++) {  
            for (int j = 0; j < 3; j++)  
                System.out.print(C[i][j] + "\t");  
            System.out.println();  
        }  
  
        // Output:  
        // 10 10 10  
        // 10 10 10  
        // 10 10 10  
    }  
}
```

Q2 Write a Java program to find the maximum of three numbers
6. using a conditional operator.

[4 Marks]

■ CONCEPT

Conditional (ternary) operator: condition ? value_if_true : value_if_false
It's a compact if-else for simple decisions.

- Nest two ternary operators to compare three numbers

■ STRUCTURED ANSWER

■ Maximum of Three Numbers:

```
public class MaxOfThree {  
    public static void main(String[] args) {  
        int a = 45, b = 78, c = 63;  
  
        // Method 1: Nested ternary  
        int max = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);  
        System.out.println("Max = " + max); // Max = 78  
  
        // Method 2: Step by step  
        int temp = (a > b) ? a : b; // max of a,b  
        int result = (temp > c) ? temp : c; // compare with c  
        System.out.println("Max = " + result); // Max = 78  
    }  
}
```

Q2
7.

Write a Java program to generate first n prime numbers.

[5 Marks]

■ CONCEPT

Prime number: divisible only by 1 and itself (2, 3, 5, 7, 11...)

Check: loop from 2 to sqrt(num) — if any divides evenly, NOT prime.

- Start from 2, count primes until we have n

■ STRUCTURED ANSWER

■ First N Prime Numbers:

```
public class PrimeNumbers {  
    static boolean isPrime(int n) {  
        if (n < 2) return false;  
        for (int i = 2; i <= Math.sqrt(n); i++)  
            if (n % i == 0) return false;  
        return true;  
    }  
    public static void main(String[] args) {  
        int n = 10, count = 0, num = 2;  
        System.out.print("First " + n + " primes: ");  
        while (count < n) {  
            if (isPrime(num)) {  
                System.out.print(num + " ");  
                count++;  
            }  
            num++;  
        }  
        // Output: 2 3 5 7 11 13 17 19 23 29  
    }  
}
```

Q2 Explain different types of operators in Java with examples:
8. arithmetic, relational, logical, assignment.

[5 Marks]

■ CONCEPT

Operators perform operations on variables/values.

- Arithmetic: + - * / %
- Relational: == != > < >= <= (return boolean)
- Logical: && || ! (combine conditions)
- Assignment: = += -= *= /= %=

■ STRUCTURED ANSWER

■ All Operator Types with Examples:

```
public class OperatorsDemo {  
    public static void main(String[] args) {  
        int a = 15, b = 4;  
  
        // Arithmetic  
        System.out.println(a + b); // 19  
        System.out.println(a - b); // 11  
        System.out.println(a * b); // 60  
        System.out.println(a / b); // 3 (integer division)  
        System.out.println(a % b); // 3 (remainder)  
  
        // Relational  
        System.out.println(a > b); // true  
        System.out.println(a == b); // false  
        System.out.println(a != b); // true  
  
        // Logical  
        System.out.println(a>10 && b<10); // true (AND)  
        System.out.println(a<10 || b<10); // true (OR)  
        System.out.println(!(a > 10)); // false (NOT)  
  
        // Assignment  
        int x = 10;  
        x += 5; System.out.println(x); // 15  
        x -= 3; System.out.println(x); // 12  
        x *= 2; System.out.println(x); // 24  
        x /= 4; System.out.println(x); // 6  
    }  
}
```

Q2 Write a Java program to reverse the digits of a number using a
9. while loop.

[4 Marks]

■ CONCEPT

Algorithm: extract last digit with % 10, build reversed number, remove digit with / 10.

- 12345 → step by step: 3,32,321,4321,54321
- Repeat while num != 0

■ STRUCTURED ANSWER

■ Reverse Digits Program:

```
public class ReverseNumber {  
    public static void main(String[] args) {  
        int num = 12345;  
        int reversed = 0;  
        int original = num;  
  
        while (num != 0) {  
            int digit = num % 10; // get last digit  
            reversed = reversed * 10 + digit; // append  
            num = num / 10; // remove last  
        }  
        System.out.println("Original : " + original); // 12345  
        System.out.println("Reversed : " + reversed); // 54321  
        // Trace: 3 -> 32 -> 321 -> 4321 -> 54321  
    }  
}
```

Q3 Explain the difference between while loop, do-while loop, and for loop in Java with examples.

[5 Marks]

■ CONCEPT

for: iterations known in advance, all control in one line.

while: condition checked BEFORE body — may never execute.

do-while: condition checked AFTER body — executes AT LEAST ONCE.

■ STRUCTURED ANSWER

■ Three Loop Types:

```
// 1. for loop – iterations known
for (int i = 1; i <= 5; i++) {
    System.out.print(i + " "); // 1 2 3 4 5
}

// 2. while loop – condition first
int n = 1;
while (n <= 5) {
    System.out.print(n + " "); // 1 2 3 4 5
    n++;
}

// 3. do-while – body first, condition after
int m = 1;
do {
    System.out.print(m + " "); // 1 2 3 4 5
    m++;
} while (m <= 5);

// do-while executes at least ONCE even if condition false:
int x = 10;
do { System.out.println("Runs once!"); } while (x < 5);
```

UNIT 2 — Inheritance, Packages & Interfaces (Q31 – Q46)

Q3
1.

What is inheritance in Java? Name any two types of inheritance.

[3 Marks]

■ CONCEPT

Inheritance: child class acquires properties and methods of parent class using 'extends'.

Promotes code reuse — write once in parent, use in all children.

- Single: one child extends one parent
- Multilevel: chain — C extends B extends A
- Hierarchical: multiple children extend one parent

■ STRUCTURED ANSWER

■ Definition + Two Types:

```
// Single Inheritance
class Animal { void eat() { System.out.println("Eating"); } }
class Dog extends Animal { void bark() { System.out.println("Woof"); } }
Dog d = new Dog(); d.eat(); d.bark(); // eat() is inherited

// Multilevel Inheritance (chain)
class A { void a() { System.out.println("A"); } }
class B extends A { void b() { System.out.println("B"); } }
class C extends B { void c() { System.out.println("C"); } }
C obj = new C();
obj.a(); obj.b(); obj.c(); // access all levels
```

Q3
2.

What is an interface in Java? How is it different from an abstract class?

[3 Marks]

■ CONCEPT

Interface: 100% abstract contract defining what a class **MUST** do (not how).

A class 'implements' interface and must provide all method bodies.

- A class can implement **MULTIPLE** interfaces (multiple inheritance of type)
- Interface: all methods abstract by default; Abstract class: can have concrete methods too

■ STRUCTURED ANSWER

■ Interface vs Abstract Class:

```
// Interface
interface Drawable { void draw(); }
interface Resizable { void resize(int factor); }

// Abstract Class
abstract class Shape {
    abstract double area(); // must override
    void display() { // concrete – already implemented
        System.out.println("Area: " + area());
    }
}

// Class extends ONE abstract class + implements MULTIPLE interfaces:
class Circle extends Shape implements Drawable, Resizable {
    double r;
    Circle(double r) { this.r = r; }
    public double area() { return Math.PI * r * r; }
    public void draw() { System.out.println("Drawing circle"); }
    public void resize(int f) { r *= f; }
}
```

Q3 What is method overriding? How is it different from method overloading?

[2 Marks]

■ CONCEPT

Overriding: child class redefines parent's method with SAME signature — runtime polymorphism.
Overloading: SAME class, same name, DIFFERENT parameters — compile-time polymorphism.

- Overriding: parent-child | Overloading: same class

■ STRUCTURED ANSWER

■ Overriding vs Overloading:

```
// Overriding – parent-child, same signature
class Animal { void speak() { System.out.println("..."); } }
class Dog extends Animal {
    @Override
    void speak() { System.out.println("Woof!"); }
}

// Overloading – same class, different params
class Printer {
    void print(int x) { System.out.println("int: " + x); }
    void print(String s){ System.out.println("str: " + s); }
}

// Runtime polymorphism (overriding):
Animal a = new Dog();
a.speak(); // Woof! – decided at RUNTIME
```

Q3
4.

What is the 'super' keyword in Java? Give one use case.

[2 Marks]

■ CONCEPT

'super' refers to the parent class. Three main uses:

- super.variable — access parent's variable
- super.method() — call parent's method
- super() — call parent's constructor (must be first line)

■ STRUCTURED ANSWER

■ super keyword in action:

```
class Animal {  
    String name = "Animal";  
    Animal(String name) { this.name = name; }  
    void sound() { System.out.println("Generic sound"); }  
}  
  
class Dog extends Animal {  
    String name = "Dog";  
    Dog(String name) {  
        super(name); // call parent constructor  
    }  
    void display() {  
        System.out.println(super.name); // parent's name  
        System.out.println(this.name); // child's name  
    }  
    void sound() {  
        super.sound(); // call parent method  
        System.out.println("Woof!");  
    }  
}
```

Q3
5.

Can a class extend multiple classes in Java? Why or why not?

[2 Marks]

■ CONCEPT

NO — Java does NOT support multiple class inheritance.

Reason: Diamond Problem — ambiguity when two parents have the same method.

- Solution: Use multiple INTERFACES (allowed in Java)

■ STRUCTURED ANSWER

■ Why Not + The Alternative:

```
// INVALID – multiple class inheritance NOT allowed:
// class C extends A, B {} // COMPILE ERROR

// DIAMOND PROBLEM example:
// class A { void show() { } }
// class B { void show() { } }
// class C extends A, B { }
// C.show() – which version? A's or B's? AMBIGUOUS!

// VALID – multiple interface implementation:
interface Flyable { void fly(); }
interface Swimmable { void swim(); }

class Duck implements Flyable, Swimmable {
    public void fly() { System.out.println("Duck flying"); }
    public void swim() { System.out.println("Duck swimming"); }
}
```

Q3 What is a package in Java? Give one example of a built-in package.
6.

[2 Marks]

■ CONCEPT

Package: a namespace that groups related classes together (like a folder).

Prevents naming conflicts, provides access control, organises large projects.

- Built-in: java.lang, java.util, java.io | User-defined: created by programmer

■ STRUCTURED ANSWER

■ Packages with Examples:

```
// Built-in packages:
import java.util.Scanner; // Scanner class from java.util
import java.util.ArrayList; // ArrayList from java.util
import java.io.FileReader; // FileReader from java.io

// java.lang is auto-imported – no need to import explicitly
// Contains: String, Math, System, Thread, Integer...

public class PackageDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println(Math.sqrt(25)); // 5.0
        System.out.println(Math.PI); // 3.14159...
    }
}
```

Q3
7.

What is an abstract class? Can it have non-abstract methods?

[2 Marks]

■ **CONCEPT**

Abstract class: cannot be instantiated directly (no objects). Designed as base class.

Can have abstract methods (no body — subclass must implement) AND concrete methods (with body).

YES — abstract class CAN have non-abstract methods

■ **STRUCTURED ANSWER**

■ **Abstract Class Example:**

```
abstract class Vehicle {
    String brand;
    Vehicle(String brand) { this.brand = brand; }

    // Abstract – MUST be implemented by subclass
    abstract void start();
    abstract double fuelEfficiency();

    // Non-abstract – already implemented, ready to use
    void fuel() { System.out.println(brand + " refuelling"); }
    void stop() { System.out.println(brand + " stopped"); }
}

class Car extends Vehicle {
    Car(String b) { super(b); }
    public void start() { System.out.println(brand+" started"); }
    public double fuelEfficiency() { return 15.5; }
}

// Vehicle v = new Vehicle(); // ERROR
Car c = new Car("Toyota"); // OK
```

Q3
8.

What is the 'final' class? Give one example.

[2 Marks]

■ **CONCEPT**

final class: cannot be extended (subclassed). Prevents inheritance.

Used to: protect class from modification, create immutable objects, ensure security.

- Java's String class is final — you cannot extend String

■ STRUCTURED ANSWER

■ final class Example:

```
// final class – cannot be subclassed
final class ImmutablePoint {
    final int x, y;
    ImmutablePoint(int x, int y) { this.x = x; this.y = y; }
    double distanceTo(ImmutablePoint other) {
        int dx = this.x - other.x;
        int dy = this.y - other.y;
        return Math.sqrt(dx*dx + dy*dy);
    }
    public String toString() { return "(" + x + "," + y + ")"; }
}

// class SubPoint extends ImmutablePoint {} // COMPILE ERROR

ImmutablePoint p1 = new ImmutablePoint(0, 0);
ImmutablePoint p2 = new ImmutablePoint(3, 4);
System.out.println(p1.distanceTo(p2)); // 5.0
```

Q3 Write a Java program to demonstrate hierarchical inheritance
9. with Animal, Dog, and Cat classes.

[6 Marks]

■ CONCEPT

Hierarchical Inheritance: ONE parent class, MULTIPLE child classes.

Dog and Cat both extend Animal — both inherit eat() and sleep().

- Each child adds its own unique behaviour

■ STRUCTURED ANSWER

■ Hierarchical Inheritance Program:

```
class Animal {
    String name;
    Animal(String name) { this.name = name; }
    void eat() { System.out.println(name + " is eating"); }
    void sleep() { System.out.println(name + " is sleeping"); }
}
class Dog extends Animal {
    Dog(String name) { super(name); }
    void bark() { System.out.println(name + ": Woof! Woof!"); }
    void fetch() { System.out.println(name + " fetches the ball"); }
}
class Cat extends Animal {
    Cat(String name) { super(name); }
    void meow() { System.out.println(name + ": Meow!"); }
    void purr() { System.out.println(name + " purrs..."); }
}
public class HierarchicalDemo {
    public static void main(String[] args) {
        Dog d = new Dog("Tommy");
        d.eat(); d.sleep(); d.bark(); d.fetch();
        Cat c = new Cat("Kitty");
        c.eat(); c.sleep(); c.meow(); c.purr();
    }
}
```

Q4 Write a Java program to demonstrate multiple inheritance using
0. interfaces with two interfaces.

[5 Marks]

■ CONCEPT

Java allows a class to implement MULTIPLE interfaces (multiple inheritance of type).
Avoids diamond problem since interfaces have no method body (before Java 8).

- class Doc implements Printable, Saveable — must implement all methods from both

■ STRUCTURED ANSWER

■ Multiple Interface Implementation:

```
interface Printable {
    void print();
}

interface Saveable {
    void save();
}

class Document implements Printable, Saveable {
    String title, content;
    Document(String t, String c) { title=t; content=c; }
    public void print() {
        System.out.println("Printing: " + title + " - " + content);
    }
    public void save() {
        System.out.println("Saving '" + title + "' to disk");
    }
}

public class MultiInterfaceDemo {
    public static void main(String[] args) {
        Document doc = new Document("Report", "Java Notes");
        doc.print(); // Printing: Report - Java Notes
        doc.save(); // Saving 'Report' to disk
        Printable p = doc; p.print(); // polymorphic ref
        Saveable s = doc; s.save();
    }
}
```

Q4 Write a Java program to demonstrate abstract class Shape with
1. Circle and Rectangle subclasses implementing area().

[6 Marks]

■ CONCEPT

Abstract Shape class declares area() without body — each shape calculates area differently.

Circle: $\text{area} = \pi * r^2$ | Rectangle: $\text{area} = \text{length} * \text{breadth}$

- Cannot create Shape object — only Circle or Rectangle objects

■ STRUCTURED ANSWER

■ Abstract Shape Program:

```
abstract class Shape {
    String color;
    Shape(String color) { this.color = color; }
    abstract double area();
    abstract double perimeter();
    void display() {
        System.out.printf("Color:%s Area:%.2f Perimeter:%.2f%n",
            color, area(), perimeter());
    }
}

class Circle extends Shape {
    double radius;
    Circle(String color, double r) { super(color); this.radius=r; }
    public double area() { return Math.PI * radius * radius; }
    public double perimeter() { return 2 * Math.PI * radius; }
}

class Rectangle extends Shape {
    double l, b;
    Rectangle(String color, double l, double b) {
        super(color); this.l=l; this.b=b;
    }
    public double area() { return l * b; }
    public double perimeter() { return 2 * (l + b); }
}

public class AbstractDemo {
    public static void main(String[] args) {
        Shape c = new Circle("Red", 7.0);
        Shape r = new Rectangle("Blue", 5.0, 3.0);
        c.display(); r.display();
    }
}
```

Q4
2.

Write a Java program to demonstrate method overriding.

[5 Marks]

■ CONCEPT

Overriding: child class provides its own implementation of parent's method.

At runtime, Java calls the child's version based on actual object type — dynamic dispatch.

- @Override annotation verifies correct overriding

■ STRUCTURED ANSWER

■ Method Overriding Program:

```
class Animal {
    void speak() { System.out.println("Animal makes a sound"); }
    void move() { System.out.println("Animal moves"); }
}

class Dog extends Animal {
    @Override void speak() { System.out.println("Dog: Woof!"); }
    @Override void move() { System.out.println("Dog runs on 4 legs"); }
}

class Bird extends Animal {
    @Override void speak() { System.out.println("Bird: Tweet!"); }
    @Override void move() { System.out.println("Bird flies"); }
}

public class OverridingDemo {
    public static void main(String[] args) {
        Animal a;
        a = new Animal(); a.speak(); // Animal makes a sound
        a = new Dog(); a.speak(); // Dog: Woof!
        a = new Bird(); a.speak(); // Bird: Tweet!
        // Runtime polymorphism – decided at runtime:
        Animal[] arr = {new Dog(), new Bird(), new Animal()};
        for(Animal x : arr) x.speak();
    }
}
```

Q4 3. Write a Java program which has a class Car. Override toString() method and create 5 instances.

[5 Marks]

■ CONCEPT

toString() from Object class returns class@hashcode by default — not useful. By overriding it, System.out.println(object) prints meaningful information.

- When you print an object, Java automatically calls toString()

■ STRUCTURED ANSWER

■ Car class with toString() override:

```
class Car {
    String brand, model, colour;
    int year; double price;

    Car(String brand, String model, String colour, int year, double price) {
        this.brand=brand; this.model=model; this.colour=colour;
        this.year=year; this.price=price;
    }
    @Override
    public String toString() {
        return "Car{" + brand + " " + model + ", " + colour +
            ", Year:" + year + ", Rs." + price + "}";
    }
}

public class CarDemo {
    public static void main(String[] args) {
        Car c1 = new Car("Toyota", "Camry", "White", 2022, 2500000);
        Car c2 = new Car("Honda", "City", "Silver", 2023, 1500000);
        Car c3 = new Car("Maruti", "Swift", "Red", 2021, 800000);
        Car c4 = new Car("Hyundai", "Creta", "Blue", 2023, 1200000);
        Car c5 = new Car("Tata", "Nexon", "Black", 2022, 950000);
        System.out.println(c1); System.out.println(c2);
        System.out.println(c3); System.out.println(c4);
        System.out.println(c5);
    }
}
```

Q4 Explain how to create and import a user-defined package in Java
4. with a complete example.

[7 Marks]

■ CONCEPT

Package groups related classes into a named namespace (like a folder).

Steps: 1) Create folder with package name 2) Add 'package pkgname;' as first line 3) Compile with -d flag 4) Import in other files

- 'package mypackage;' must be FIRST statement in the file

■ STRUCTURED ANSWER

■ Step 1 — Create package file (mypackage/MathOperations.java):

```
package mypackage;

public class MathOperations {
    public int add(int a, int b) { return a + b; }
    public int subtract(int a, int b) { return a - b; }
    public int multiply(int a, int b) { return a * b; }
    public double divide(double a, double b) {
        if(b==0) throw new ArithmeticException("Div by 0");
        return a / b;
    }
}
```

■ Step 2 — Compile: javac -d . mypackage/MathOperations.java

■ Step 3 — Use the package (Main.java):

```
import mypackage.MathOperations; // import specific class
// import mypackage.*; // or import all

public class Main {
    public static void main(String[] args) {
        MathOperations m = new MathOperations();
        System.out.println(m.add(10, 5)); // 15
        System.out.println(m.subtract(10, 5)); // 5
        System.out.println(m.multiply(10, 5)); // 50
        System.out.println(m.divide(10, 5)); // 2.0
    }
}

// Compile: javac Main.java
// Run: java Main
```

Q4
5.

Write a Java program to demonstrate multilevel inheritance.

[5 Marks]

■ CONCEPT

Multilevel: A chain of inheritance — Grandparent → Parent → Child.
Each level inherits from the level above.

- Dog object has access to Animal + Mammal + Dog methods

■ STRUCTURED ANSWER

■ Multilevel Inheritance Program:

```
class Animal {
    void breathe() { System.out.println("Breathing"); }
    void move() { System.out.println("Moving"); }
}
class Mammal extends Animal {
    void feedYoung() { System.out.println("Feeding young with milk"); }
    void warmBlooded() { System.out.println("Warm blooded"); }
}
class Dog extends Mammal {
    void bark() { System.out.println("Woof!"); }
    void fetch() { System.out.println("Fetching ball"); }
}
public class MultilevelDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.breathe(); // from Animal (level 1)
        d.move(); // from Animal
        d.feedYoung(); // from Mammal (level 2)
        d.warmBlooded();
        d.bark(); // Dog's own (level 3)
        d.fetch();
    }
}
```

Q4 Explain access modifiers in Java: public, private, protected, and default with examples.

[5 Marks]

■ CONCEPT

Access modifiers control where class members (variables/methods) can be accessed from.

- public: everywhere | private: same class only
- protected: same package + subclasses | default: same package only

■ STRUCTURED ANSWER

■ All Four Access Modifiers:

```
public class AccessDemo {  
    public int pubVar = 1; // accessible everywhere  
    protected int proVar = 2; // same package + subclasses  
    int defVar = 3; // same package only (no keyword)  
    private int priVar = 4; // this class only  
  
    public void publicMethod() { System.out.println("public"); }  
    protected void protectedMethod() { System.out.println("protected"); }  
    void defaultMethod() { System.out.println("default"); }  
    private void privateMethod() { System.out.println("private"); }  
  
    // Getter/setter for private (encapsulation pattern):  
    public int getPriVar() { return priVar; }  
    public void setPriVar(int v) { priVar = v; }  
}  
  
// Summary:  
// Modifier Same Class Same Pkg Subclass Other  
// public YES YES YES YES  
// protected YES YES YES NO  
// default YES YES NO NO  
// private YES NO NO NO
```

UNIT 3 — Exception Handling, Multithreading & Collections (Q47 – Q64)

Q4 7. What is an exception in Java? Difference between checked and unchecked exceptions?

[3 Marks]

■ CONCEPT

Exception: unexpected event during program execution that disrupts normal flow.

Checked: compiler FORCES handling — recoverable (IOException, FileNotFoundException)

Unchecked: runtime errors, usually programming bugs (NullPointerException, ArrayIndexOutOfBoundsException)

■ STRUCTURED ANSWER

■ Checked vs Unchecked Exceptions:

```
// Checked Exception – must handle with try-catch or throws
import java.io.*;
try {
    FileReader f = new FileReader("data.txt"); // may throw
} catch(FileNotFoundException e) {
    System.out.println(e.getMessage());
}

// Unchecked Exceptions – runtime bugs
int[] arr = {1, 2, 3};
System.out.println(arr[10]); // ArrayIndexOutOfBoundsException

String s = null;
s.length(); // NullPointerException

int x = 5 / 0; // ArithmeticException

// Hierarchy:
// Throwable -> Error (JVM errors)
// -> Exception -> Checked (IOException, etc.)
// -> RuntimeException (unchecked)
```

Q4 8. What is a user-defined exception? Give a real-life scenario.

[3 Marks]

■ CONCEPT

User-defined exception: custom exception class created by programmer for specific app errors.

Created by extending Exception (checked) or RuntimeException (unchecked).

Real-life scenario: Voting System — person below 18 should throw AgeNotValidException

■ STRUCTURED ANSWER

■ User-Defined Exception Example:

```
class AgeNotValidException extends Exception {
    AgeNotValidException(String msg) { super(msg); }
}

class VotingSystem {
    void checkEligibility(int age) throws AgeNotValidException {
        if (age < 18)
            throw new AgeNotValidException(
                "Age " + age + " invalid. Must be 18+");
        System.out.println("Age " + age + " – Eligible to vote!");
    }
}

public class VoteDemo {
    public static void main(String[] args) {
        VotingSystem vs = new VotingSystem();
        try {
            vs.checkEligibility(25); // Eligible
            vs.checkEligibility(15); // throws exception
        } catch (AgeNotValidException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Q4 **What is multithreading? What is the advantage of using threads?**

[2 Marks]

■ CONCEPT

Multithreading: multiple threads (execution units) running concurrently within one program. Each thread runs independently but shares process memory.

- Advantages: better CPU use, faster programs, responsive UI, resource sharing

■ STRUCTURED ANSWER

■ Multithreading Basics:

```
class Task extends Thread {
    String name;
    Task(String name) { this.name = name; }
    public void run() {
        for(int i=1; i<=3; i++)
            System.out.println(name + " step " + i);
    }
}

public class ThreadDemo {
    public static void main(String[] args) {
        Task t1 = new Task("Download");
        Task t2 = new Task("Upload");
        t1.start(); t2.start(); // both run concurrently
    }
}

// Advantages:
// 1. CPU utilisation – no idle time during I/O
// 2. Performance – parallel execution
// 3. Responsiveness – UI stays active
// 4. Resource sharing – threads share memory
```

Q5 What is the difference between extending Thread class and
0. implementing Runnable interface?

[2 Marks]

■ CONCEPT

Extending Thread: simpler, but class cannot extend any other class (single inheritance).

Implementing Runnable: preferred — class can still extend another class.

- Runnable promotes better OOP design (separation of task from thread)

■ STRUCTURED ANSWER

■ Thread vs Runnable:

```
// Method 1: Extend Thread class
class MyThread extends Thread {
    public void run() { System.out.println("Thread: " + getName()); }
}

MyThread t1 = new MyThread();
t1.start();

// Method 2: Implement Runnable (PREFERRED)
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable: " + Thread.currentThread().getName());
    }
}

Thread t2 = new Thread(new MyRunnable());
t2.start();

// Lambda (Java 8+):
Thread t3 = new Thread(() -> System.out.println("Lambda!"));
t3.start();
```

Q5
1.

What is thread priority? How is it set in Java?

[2 Marks]

■ CONCEPT

Thread priority: hint to scheduler about importance (1 to 10, higher = more CPU time).

Not guaranteed — OS/JVM decides actual execution.

- MIN_PRIORITY=1, NORM_PRIORITY=5 (default), MAX_PRIORITY=10

■ STRUCTURED ANSWER

■ Thread Priority Example:

```
class PriorityDemo extends Thread {  
    PriorityDemo(String name) { super(name); }  
    public void run() {  
        System.out.println(getName() + " priority: " + getPriority());  
    }  
}  
  
public class PriorityTest {  
    public static void main(String[] args) {  
        PriorityDemo t1 = new PriorityDemo("Low");  
        PriorityDemo t2 = new PriorityDemo("Normal");  
        PriorityDemo t3 = new PriorityDemo("High");  
        t1.setPriority(Thread.MIN_PRIORITY); // 1  
        t2.setPriority(Thread.NORM_PRIORITY); // 5  
        t3.setPriority(Thread.MAX_PRIORITY); // 10  
        t1.start(); t2.start(); t3.start();  
        // Note: higher priority gets MORE CPU time (not guaranteed)  
    }  
}
```

Q5
2.

What is the Collections Framework in Java?

[2 Marks]

■ CONCEPT

Collections Framework: unified architecture for storing and manipulating groups of objects.

Provides ready-made data structures: List, Set, Map, Queue.

- No need to implement data structures from scratch

■ STRUCTURED ANSWER

■ Collections Framework Overview:

```
// Core interfaces and their implementations:

// List (ordered, duplicates allowed):
List list = new ArrayList<>(); // dynamic array
List ll = new LinkedList<>(); // doubly linked list

// Set (no duplicates):
Set hs = new HashSet<>(); // no order
Set ts = new TreeSet<>(); // sorted order

// Map (key-value pairs):
Map hm = new HashMap<>(); // no order
Map tm = new TreeMap<>(); // sorted by key

// Queue (FIFO):
Queue pq = new PriorityQueue<>();

// Utility algorithms:
Collections.sort(list);
Collections.shuffle(list);
Collections.max(list);
```

Q5
3.

What is the difference between ArrayList and LinkedList?

[2 Marks]

■ CONCEPT

ArrayList: dynamic array, fast random access $O(1)$, slow insert/delete at middle $O(n)$.

LinkedList: doubly linked list, slow random access $O(n)$, fast insert/delete $O(1)$ with iterator.

- Use ArrayList for read-heavy; LinkedList for frequent insert/delete

■ STRUCTURED ANSWER

■ ArrayList vs LinkedList:

```
import java.util.*;

// ArrayList – fast get, slow insert/delete at middle
ArrayList al = new ArrayList<>();
al.add(10); al.add(20); al.add(30);
System.out.println(al.get(1)); // 20 – O(1) fast
al.add(1, 15); // O(n) – must shift all elements

// LinkedList – fast insert/delete, slow random access
LinkedList ll = new LinkedList<>();
ll.add(30); ll.add(10); ll.add(20);
ll.addFirst(5); // O(1) – just update head pointer
ll.addLast(40); // O(1) – just update tail pointer
ll.removeFirst(); // O(1)

// LinkedList as Queue:
Queue queue = new LinkedList<>();
queue.offer("First"); queue.offer("Second");
System.out.println(queue.poll()); // First (FIFO)
```

Q5
4.

What is a HashSet? How is it different from ArrayList?

[2 Marks]

■ CONCEPT

HashSet: implements Set interface. No duplicates, no guaranteed order, backed by HashMap.

ArrayList: implements List interface. Duplicates allowed, insertion order maintained.

- HashSet: uniqueness + fast lookup | ArrayList: ordered + indexed access

■ STRUCTURED ANSWER

■ HashSet vs ArrayList:

```
import java.util.*;

// ArrayList – duplicates allowed, ordered
ArrayList al = new ArrayList<>();
al.add("Java"); al.add("Python"); al.add("Java");
System.out.println(al); // [Java, Python, Java] – duplicate kept

// HashSet – no duplicates, no order guarantee
HashSet hs = new HashSet<>();
hs.add("Java"); hs.add("Python"); hs.add("Java");
System.out.println(hs); // [Java, Python] – duplicate ignored

// Speed comparison:
System.out.println(hs.contains("Java")); // O(1) – hash lookup
System.out.println(al.contains("Java")); // O(n) – linear search

// Use case: remove duplicates from a list:
HashSet unique = new HashSet<>(al); // removes dups
```

Q5 Explain try-catch-finally block in Java with a simple example.
5. What is the role of the finally block?

[5 Marks]

■ CONCEPT

try: code that might throw exception. catch: handles the exception. finally: ALWAYS runs.
Finally runs whether exception occurs or not, whether caught or not.

- Used for cleanup: closing files, database connections, releasing resources

■ STRUCTURED ANSWER

■ try-catch-finally Complete Example:

```
public class TryCatchDemo {  
    public static void main(String[] args) {  
        // Example 1: exception occurs  
        try {  
            System.out.println("try: starting");  
            int result = 10 / 0; // ArithmeticException  
            System.out.println("never reached");  
        } catch(ArithmeticException e) {  
            System.out.println("catch: " + e.getMessage()); // / by zero  
        } finally {  
            System.out.println("finally: ALWAYS executes");  
        }  
  
        // Example 2: multiple catch blocks  
        try {  
            int[] arr = new int[3];  
            arr[10] = 5; // ArrayIndexOutOfBoundsException  
        } catch(ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array error: " + e.getMessage());  
        } catch(Exception e) {  
            System.out.println("General error");  
        } finally {  
            System.out.println("Cleanup complete");  
        }  
    }  
}
```

Q5
6.

Write a Java program to demonstrate multithreading: Thread1 prints 'Hello' every 1000ms, Thread2 prints 'World' every 2000ms.

[6 Marks]

■ CONCEPT

Two threads run concurrently with different sleep intervals.
Thread.sleep(ms) pauses the thread without blocking others.

- Thread1 prints Hello twice for every World from Thread2

■ STRUCTURED ANSWER

■ Two Threads with Different Intervals:

```
class HelloThread extends Thread {
    public void run() {
        for(int i = 0; i < 5; i++) {
            System.out.println("Thread1: Hello");
            try { Thread.sleep(1000); }
            catch(InterruptedException e) { e.printStackTrace(); }
        }
    }
}

class WorldThread extends Thread {
    public void run() {
        for(int i = 0; i < 5; i++) {
            System.out.println("Thread2: World");
            try { Thread.sleep(2000); }
            catch(InterruptedException e) { e.printStackTrace(); }
        }
    }
}

public class MultiThreading {
    public static void main(String[] args) {
        HelloThread t1 = new HelloThread();
        WorldThread t2 = new WorldThread();
        t1.start(); // both run concurrently
        t2.start();
    }
}

// Output: Hello, Hello, World, Hello, Hello, World...
```

Q5 Write a Java program to create `InsufficientFundsException` for a
7. banking program.

[6 Marks]

■ CONCEPT

Custom exception for banking: when withdrawal > balance, throw `InsufficientFundsException`.
The exception carries meaningful message with balance and deficit amount.

- Extend `Exception` for checked exception — caller must handle it

■ STRUCTURED ANSWER

■ Banking with Custom Exception:

```
class InsufficientFundsException extends Exception {
    private double deficit;
    InsufficientFundsException(double deficit) {
        super("Insufficient funds. Short by Rs." + deficit);
        this.deficit = deficit;
    }
    double getDeficit() { return deficit; }
}

class BankAccount {
    private String owner;
    private double balance;
    BankAccount(String o, double b) { owner=o; balance=b; }
    void deposit(double a) {
        balance += a;
        System.out.println("Deposited Rs."+a+". Balance: "+balance);
    }
    void withdraw(double a) throws InsufficientFundsException {
        if(a > balance) throw new InsufficientFundsException(a-balance);
        balance -= a;
        System.out.println("Withdrawn Rs."+a+". Balance: "+balance);
    }
}

public class BankApp {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("Alice", 5000);
        try {
            acc.deposit(2000); // 7000
            acc.withdraw(3000); // 4000
            acc.withdraw(10000); // throws!
        } catch(InsufficientFundsException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Q5 Write a Java program to develop a user-defined exception for
8. 'Divide by Zero' error.

[5 Marks]

■ CONCEPT

Create DivideByZeroException extending ArithmeticException.

In divide() method: if denominator is 0, throw the custom exception.

- Custom exception gives more descriptive error than built-in ArithmeticException

■ STRUCTURED ANSWER

■ Custom Divide by Zero Exception:

```
class DivideByZeroException extends ArithmeticException {
    DivideByZeroException() {
        super("Cannot divide by zero!");
    }
    DivideByZeroException(String msg) { super(msg); }
}

class Calculator {
    double divide(double a, double b) {
        if(b == 0)
            throw new DivideByZeroException(
                "Attempted " + a + " / 0 - undefined!");
        return a / b;
    }
}

public class DivideDemo {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        try {
            System.out.println(calc.divide(10, 2)); // 5.0
            System.out.println(calc.divide(10, 0)); // throws!
        } catch(DivideByZeroException e) {
            System.out.println("Caught: " + e.getMessage());
        }
    }
}
```

Q5 Write a Java program to develop a Banking Application that
9. throws 'Not Sufficient Fund' exception.

[6 Marks]

■ CONCEPT

Complete banking app: custom NotSufficientFundException + Account class.

Exception thrown when withdrawal amount exceeds current balance.

- Shows available balance and requested amount in the error message

■ STRUCTURED ANSWER

■ Banking Application with NotSufficientFundException:

```
class NotSufficientFundException extends Exception {
    NotSufficientFundException(double balance, double requested) {
        super("Not Sufficient Fund! Available: Rs." + balance +
            ", Requested: Rs." + requested);
    }
}

class Account {
    private String accNo, holder;
    private double balance;
    Account(String a, String h, double b) {
        accNo=a; holder=h; balance=b;
    }
    void deposit(double a) {
        balance += a;
        System.out.println(holder+" deposited Rs."+a+". Balance: "+balance);
    }
    void withdraw(double a) throws NotSufficientFundException {
        if(a > balance)
            throw new NotSufficientFundException(balance, a);
        balance -= a;
        System.out.println(holder+" withdrew Rs."+a+". Balance: "+balance);
    }
    void showBalance() {
        System.out.println(holder+"[" + accNo + "] Balance: Rs." + balance);
    }
}

public class BankingApp {
    public static void main(String[] args) {
        Account a = new Account("ACC001","Rahul",10000);
        a.showBalance();
        try {
            a.deposit(5000);
            a.withdraw(8000);
            a.withdraw(20000); // throws!
        } catch(NotSufficientFundException e) {
            System.out.println("Error: " + e.getMessage());
        }
        a.showBalance();
    }
}
```

Q6
0.

Explain the life cycle of a thread in Java with all five states.

[5 Marks]

■ **CONCEPT**

Thread lifecycle has 5 states:

1. NEW — created, start() not called
2. RUNNABLE — start() called, ready/running
3. BLOCKED — waiting for lock
4. TIMED_WAITING — sleep(ms) or wait(timeout)
5. TERMINATED — run() completed

■ **STRUCTURED ANSWER**

■ **Thread Life Cycle with Code:**

```
class LifeCycle extends Thread {
    public void run() {
        System.out.println("State: RUNNABLE");
        try {
            Thread.sleep(2000); // State: TIMED_WAITING for 2s
            System.out.println("State: RUNNABLE again");
        } catch (InterruptedException e) { e.printStackTrace(); }
        System.out.println("About to TERMINATE");
    }
}

public class ThreadLifeCycle {
    public static void main(String[] args) throws Exception {
        LifeCycle t = new LifeCycle();
        System.out.println(t.getState()); // NEW
        t.start();
        System.out.println(t.getState()); // RUNNABLE
        Thread.sleep(500);
        System.out.println(t.getState()); // TIMED_WAITING
        t.join(); // wait for t to finish
        System.out.println(t.getState()); // TERMINATED
    }
}

// States: NEW -> RUNNABLE -> TIMED_WAITING -> RUNNABLE -> TERMINATED
```

Q6
1.

Explain the Collections Framework. Describe ArrayList, LinkedList, HashSet, and HashMap with use cases.

[7 Marks]

■ CONCEPT

Collections Framework: ready-made data structures in java.util.

- ArrayList: dynamic array | LinkedList: doubly linked list
- HashSet: unique elements | HashMap: key-value pairs

All support generics for type safety

■ STRUCTURED ANSWER

■ All Four Collections with Examples:

```
import java.util.*;

public class CollectionsDemo {
    public static void main(String[] args) {
        // ArrayList – ordered, duplicates OK, fast access
        ArrayList al = new ArrayList<>();
        al.add("Alice"); al.add("Bob"); al.add("Alice");
        System.out.println("ArrayList: " + al); // [Alice, Bob, Alice]
        Collections.sort(al);
        System.out.println("Sorted: " + al); // [Alice, Alice, Bob]

        // LinkedList – fast insert/delete, works as Queue
        LinkedList ll = new LinkedList<>();
        ll.add(30); ll.addFirst(5); ll.addLast(40);
        System.out.println("LinkedList: " + ll); // [5, 30, 40]
        ll.removeFirst(); // 0(1)

        // HashSet – unique, no order, fast lookup
        HashSet hs = new HashSet<>();
        hs.add("Java"); hs.add("Python"); hs.add("Java");
        System.out.println("HashSet: " + hs); // [Java, Python]

        // HashMap – key-value, fast lookup by key
        HashMap map = new HashMap<>();
        map.put("Alice", 92); map.put("Bob", 85); map.put("Carol", 78);
        System.out.println("Map: " + map);
        for(Map.Entry e : map.entrySet())
            System.out.println(e.getKey() + " -> " + e.getValue());
    }
}
```

Q6 Write a Java program to perform read and write operations on a
2. text file.

[7 Marks]

■ CONCEPT

File I/O in Java uses streams. Key classes:

- FileWriter/BufferedWriter: write text to file
- FileReader/BufferedReader: read text from file
- Always handle IOException, always close streams (try-with-resources)

■ STRUCTURED ANSWER

■ Complete File Read/Write Program:

```
import java.io.*;

public class FileOperations {

    // WRITE to file

    static void writeFile(String filename) {
        try (BufferedWriter bw = new BufferedWriter(
            new FileWriter(filename))) {
            bw.write("Line 1: Hello Java!");
            bw.newLine();
            bw.write("Line 2: File I/O Demo");
            bw.newLine();
            bw.write("Line 3: BufferedWriter is efficient");
            System.out.println("File written: " + filename);
        } catch(IOException e) {
            System.out.println("Write error: " + e.getMessage());
        }
    }

    // READ from file

    static void readFile(String filename) {
        System.out.println("\nReading: " + filename);
        try (BufferedReader br = new BufferedReader(
            new FileReader(filename))) {
            String line; int no = 1;
            while((line = br.readLine()) != null)
                System.out.println(no++ + ": " + line);
        } catch(FileNotFoundException e) {
            System.out.println("File not found!");
        } catch(IOException e) {
            System.out.println("Read error: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        writeFile("output.txt");
        readFile("output.txt");
    }
}
```

Q6 3. Write a Java program using ArrayList to store and display five student names.

[4 Marks]

■ **CONCEPT**

ArrayList stores ordered list of names with duplicates allowed.
Add with add(), access with get(index) or for-each loop.

- Type parameter enforces type safety

■ **STRUCTURED ANSWER**

■ **ArrayList of Five Student Names:**

```
import java.util.ArrayList;

public class StudentNames {

    public static void main(String[] args) {
        ArrayList students = new ArrayList<>();

        // Add 5 student names
        students.add("Arjun Patel");
        students.add("Priya Shah");
        students.add("Rahul Verma");
        students.add("Sneha Joshi");
        students.add("Karan Mehta");

        // Display using for-each
        System.out.println("Students (" + students.size() + "):");
        for(String name : students)
            System.out.println(" " + name);

        // Access by index
        System.out.println("First: " + students.get(0));
        System.out.println("Last: " + students.get(4));

        // Search
        System.out.println("Has Rahul? " + students.contains("Rahul Verma"));
    }
}
```

Q6 4. Write a Java program using HashMap to store and display student name and marks.

[5 Marks]

■ **CONCEPT**

HashMap: student name is key, marks is value. Keys are unique.
Add with put(), retrieve with get(key), iterate with entrySet().

- Fast O(1) lookup by student name

■ STRUCTURED ANSWER

■ HashMap of Student Marks:

```
import java.util.*;

public class StudentMarks {
    public static void main(String[] args) {
        HashMap marks = new HashMap<>();

        // Add students and marks
        marks.put("Arjun", 88);
        marks.put("Priya", 92);
        marks.put("Rahul", 75);
        marks.put("Sneha", 95);
        marks.put("Karan", 83);

        // Display all
        System.out.println("Student Marks:");
        for(Map.Entry e : marks.entrySet())
            System.out.printf("%-8s : %d\n", e.getKey(), e.getValue());

        // Access specific student
        System.out.println("Priya's marks: " + marks.get("Priya"));

        // Update
        marks.put("Rahul", 80);
        System.out.println("Updated Rahul: " + marks.get("Rahul"));

        // Find topper
        String topper = Collections.max(marks.entrySet(),
            Map.Entry.comparingByValue()).getKey();
        System.out.println("Topper: " + topper + " (" + marks.get(topper) + ")");
    }
}
```